Experiment : 6

❖ **Aim :**

To design and develop a Contact Management System using that allows users to add, view, update, and delete contact details.

❖ **Tasks :**

• Project Setup & Introduction – Create a project folder named contact_management_system, set up a project and app, and create required folders such as templates and static.

• Django App Initialization – Configure the app in settings, create views, and set up URLs to display contacts on the homepage.

• Add New Contacts – Create a form to add new contacts with Name, Phone Number, and Email, and store data in the database.

• View & Display Contacts – Display all contacts in a table or list using templates with proper formatting and navigation.

• Update Contact Details – Provide an edit option to update existing contact details with pre-filled form data.

• Delete Contacts – Add a delete option to remove contacts from the database and update the list dynamically.

• Search & Styling (Bonus) – Add search functionality by name or phone number and apply basic CSS for better UI design.

❖ **Tech Stack Used :**

• Python • • HTML • CSS •

❖ Html Code :

➤Index.html :

```
{% extends "base.html" %}
{% block title %}All Contacts{% endblock %}

{% block content %}
<div class="page-header">
<div>
<h1 class="page-title">All Contacts</h1>
<p style="color:var(--muted); font-size:0.9rem; margin-top:0.25rem;">
{{ contacts|length }} contact{% if contacts|length != 1 %}s{% endif %}
```

```
{% if query %} matching "<strong style="color:var(--accent)">{{ query }}</strong>"{%
endif %}
</p>
</div>

<div style="display:flex; gap:0.75rem; align-items:center; flex-wrap:wrap;">
<!-- Search form -->
<form method="GET" action="{{ url_for('index') }}" class="search-bar">
<input
type="text"
name="q"
value="{{ query }}"
placeholder="Search name or phone..."
aria-label="Search contacts"
/>
<button type="submit" class="btn btn-secondary">🔍</button>
{% if query %}
<a href="{{ url_for('index') }}" class="btn btn-secondary" title="Clear search">✕</a>
{% endif %}
</form>

<a href="{{ url_for('add_contact') }}" class="btn btn-primary">＋ Add Contact</a>
</div>
</div>

<div class="card">
{% if contacts %}
<div class="table-wrap">
<table>
<thead>
<tr>
<th>#</th>
<th>Name</th>
<th>Phone</th>
<th>Email</th>
<th>Address</th>
<th>Actions</th>
</tr>
</thead>
<tbody>
{% for contact in contacts %}
<tr>
<td><span class="badge">{{ loop.index }}</span></td>
<td><strong>{{ contact.name }}</strong></td>
<td>{{ contact.phone }}</td>
<td>
<a href="mailto:{{ contact.email }}"
```

```
style="color:var(--accent2); text-decoration:none;">
{{ contact.email }}
</a>
</td>
<td style="color:var(--muted)">{{ contact.address or '—' }}</td>
<td>
<div class="actions">
<a href="{{ url_for('edit_contact', contact_id=contact.id) }}"
class="btn btn-edit">✏ Edit</a>
<form method="POST"
action="{{ url_for('delete_contact', contact_id=contact.id) }}"
onsubmit="return confirm('Delete {{ contact.name }}?');">
<button type="submit" class="btn btn-danger">🗑 Delete</button>
</form>
</div>
</td>
</tr>
{% endfor %}
</tbody>
</table>
</div>
{% else %}
<div class="empty-state">
<div class="icon">📌</div>
{% if query %}
<p>No contacts match "<strong>{{ query }}</strong>".</p>
<a href="{{ url_for('index') }}" class="btn btn-secondary">Clear Search</a>
{% else %}
<p>No contacts yet. Add your first one!</p>
<a href="{{ url_for('add_contact') }}" class="btn btn-primary">＋ Add Contact</a>
{% endif %}
</div>
{% endif %}
</div>
{% endblock %}
```

➤**edit_contact.html :**

```
{% extends "base.html" %}
{% block title %}Edit Contact{% endblock %}

{% block content %}
<div class="page-header">
<h1 class="page-title">Edit Contact</h1>
<a href="{{ url_for('index') }}" class="btn btn-secondary">← Back</a>
</div>
```

```
<div class="card" style="max-width:560px;">
<p style="color:var(--muted); font-size:0.85rem; margin-bottom:1.4rem;">
Editing: <strong style="color:var(--text)">{{ contact.name }}</strong>
 <span class="badge">ID {{ contact.id }}</span>
</p>

<form method="POST"
action="{{ url_for('edit_contact', contact_id=contact.id) }}">

<div class="form-group">
<label for="name">Full Name *</label>
<input
type="text"
id="name"
name="name"
value="{{ form_data.get('name', contact.name) }}"
required
/>
</div>

<div class="form-group">
<label for="phone">Phone Number *</label>
<input
type="tel"
id="phone"
name="phone"
value="{{ form_data.get('phone', contact.phone) }}"
required
/>
</div>

<div class="form-group">
<label for="email">Email Address *</label>
<input
type="email"
id="email"
name="email"
value="{{ form_data.get('email', contact.email) }}"
required
/>
</div>
```

```
<div class="form-group">
<label for="address">Address <span style="color:var(--muted); font-
weight:400;">(optional)</span></label>
<input
type="text"
id="address"
name="address"
value="{{ form_data.get('address', contact.address) }}"
/>
</div>

<div class="form-actions">
<button type="submit" class="btn btn-primary">💾 Update
Contact</button>
<a href="{{ url_for('index') }}" class="btn btn-secondary">Cancel</a>
</div>

</form>
</div>
{% endblock %}
```

➤**Base.html :**

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1.0"
/>
<title>{% block title %}Contacts{% endblock %} | ContactVault</title>
<link rel="preconnect" href="https://fonts.googleapis.com" />
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
<link
href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@300;40
0;500;600&family=Syne:wght@700;800&display=swap" rel="stylesheet"
/>
<style>
/* — Reset & Variables ———————————————— */
*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }

:root {
--bg:        #0d0f14;
--surface:   #161921;
--surface2:  #1e2130;
--border:    #2a2f45;
--accent:    #6ee7b7;   /* mint green */
--accent2:   #38bdf8;   /* sky blue */
--danger:    #f87171;
--warn:      #fbbf24;
--text:      #e2e8f0;
--muted:     #64748b;
--radius:    12px;
--shadow:    0 4px 24px rgba(0,0,0,0.5);
}
```

```
body {
font-family: 'DM Sans', sans-serif;
background: var(--bg);
color: var(--text);
min-height: 100vh;
display: flex;
flex-direction: column;
}

/* — Nav ———————————————————————————————— */
nav {
background: var(--surface);
border-bottom: 1px solid var(--border);
padding: 0 2rem;
display: flex;
align-items: center;
justify-content: space-between;
height: 64px;
position: sticky;
top: 0;
z-index: 100;
}

.nav-brand {
font-family: 'Syne', sans-serif;
font-size: 1.35rem;
font-weight: 800;
letter-spacing: -0.5px;
color: var(--accent);
text-decoration: none;
}

.nav-brand span { color: var(--text); }
```

```
.nav-links { display: flex; gap: 0.5rem; align-items: center; }

.btn {
display: inline-flex;
align-items: center;
gap: 0.4rem;
padding: 0.45rem 1rem;
border-radius: 8px;
font-family: 'DM Sans', sans-serif;
font-size: 0.875rem;
font-weight: 500;
cursor: pointer;
border: none;
text-decoration: none;
transition: all 0.18s ease;
}

.btn-primary {
background: var(--accent);
color: #0d0f14;
}
.btn-primary:hover { background: #a7f3d0; transform: translateY(-1px); }

.btn-secondary {
background: var(--surface2);
color: var(--text);
border: 1px solid var(--border);
}
.btn-secondary:hover { background: var(--border); }

.btn-danger {
background: transparent;
```

```
color: var(--danger);
border: 1px solid var(--danger);
}
.btn-danger:hover { background: var(--danger); color: #fff; }

.btn-edit {
background: transparent;
color: var(--accent2);
border: 1px solid var(--accent2);
}
.btn-edit:hover { background: var(--accent2); color: #fff; }

/* — Flash Messages ———————————————————— */
.flash-container {
max-width: 960px;
margin: 1.5rem auto 0;
padding: 0 1.5rem;
width: 100%;
}

.flash {
padding: 0.75rem 1.1rem;
border-radius: 8px;
font-size: 0.9rem;
margin-bottom: 0.6rem;
font-weight: 500;
}
.flash.success { background: #052e16; border: 1px solid #16a34a; color:
#86efac; }
.flash.error   { background: #2d0a0a; border: 1px solid #dc2626; color:
#fca5a5; }

/* — Main ———————————————————————————— */
main {
```

```
max-width: 960px;
margin: 2rem auto;
padding: 0 1.5rem;
flex: 1;
width: 100%;
}

/* — Page header ———————————————————————— */
.page-header {
display: flex;
align-items: center;
justify-content: space-between;
margin-bottom: 1.8rem;
gap: 1rem;
flex-wrap: wrap;
}

.page-title {
font-family: 'Syne', sans-serif;
font-size: 1.8rem;
font-weight: 800;
letter-spacing: -0.5px;
}

/* — Card ———————————————————————————— */
.card {
background: var(--surface);
border: 1px solid var(--border);
border-radius: var(--radius);
padding: 1.8rem;
box-shadow: var(--shadow);
}
```

```
/* — Table ———————————————————————————— */
.table-wrap { overflow-x: auto; }

table {
width: 100%;
border-collapse: collapse;
font-size: 0.9rem;
}

thead tr {
background: var(--surface2);
border-bottom: 2px solid var(--border);
}

th {
padding: 0.75rem 1rem;
text-align: left;
font-weight: 600;
color: var(--muted);
text-transform: uppercase;
font-size: 0.75rem;
letter-spacing: 0.07em;
}

td {
padding: 0.85rem 1rem;
border-bottom: 1px solid var(--border);
vertical-align: middle;
}

tbody tr:last-child td { border-bottom: none; }

tbody tr:hover { background: var(--surface2); }
```

```
.actions { display: flex; gap: 0.5rem; }

/* — Form ———————————————————————————— */
.form-group { margin-bottom: 1.2rem; }

label {
display: block;
font-size: 0.825rem;
font-weight: 600;
color: var(--muted);
text-transform: uppercase;
letter-spacing: 0.05em;
margin-bottom: 0.4rem;
}

input[type="text"],
input[type="email"],
input[type="tel"] {
width: 100%;
padding: 0.65rem 0.9rem;
background: var(--surface2);
border: 1px solid var(--border);
border-radius: 8px;
color: var(--text);
font-family: 'DM Sans', sans-serif;
font-size: 0.95rem;
outline: none;
transition: border-color 0.15s;
}

input:focus { border-color: var(--accent); }
```

```
.form-actions {
display: flex;
gap: 0.75rem;
margin-top: 1.5rem;
}

/* — Empty state ———————————————————— */
.empty-state {
text-align: center;
padding: 3rem 1rem;
color: var(--muted);
}
.empty-state .icon { font-size: 3rem; margin-bottom: 0.75rem; }
.empty-state p { margin-bottom: 1.2rem; }

/* — Search bar ———————————————————— */
.search-bar {
display: flex;
gap: 0.5rem;
flex: 1;
min-width: 220px;
max-width: 360px;
}

.search-bar input {
flex: 1;
padding: 0.55rem 0.9rem;
background: var(--surface2);
border: 1px solid var(--border);
border-radius: 8px;
color: var(--text);
font-family: 'DM Sans', sans-serif;
font-size: 0.9rem;
```

```
outline: none;
}
.search-bar input:focus { border-color: var(--accent); }

/* — Footer ———————————————————————————— */
footer {
text-align: center;
padding: 1.5rem;
color: var(--muted);
font-size: 0.8rem;
border-top: 1px solid var(--border);
}

/* — Badge ———————————————————————————— */
.badge {
display: inline-block;
background: var(--surface2);
border: 1px solid var(--border);
color: var(--accent);
font-size: 0.75rem;
font-weight: 600;
padding: 0.2rem 0.6rem;
border-radius: 99px;
}
</style>
</head>
<body>

<nav>
<a href="{{ url_for('index') }}" class="nav-
brand">Contact<span>Vault</span></a>
<div class="nav-links">
<a href="{{ url_for('index') }}" class="btn btn-secondary">🏠 Home</a>
```

```
<a href="{{ url_for('add_contact') }}" class="btn btn-primary">＋ Add
Contact</a>
</div>
</nav>

<!-- Flash messages -->
{% with messages = get_flashed_messages(with_categories=true) %}
{% if messages %}
<div class="flash-container">
{% for category, message in messages %}
<div class="flash {{ category }}">{{ message }}</div>
{% endfor %}
</div>
{% endif %}
{% endwith %}

<main>
{% block content %}{% endblock %}
</main>

<footer>
ContactVault &mdash; Contact Management System &copy; 2026
</footer>

</body>
</html>
```

➤Add Contact.html :

```
{% extends "base.html" %}
{% block title %}Add Contact{% endblock %}

{% block content %}
<div class="page-header">
```

```
<h1 class="page-title">Add New Contact</h1>
<a href="{{ url_for('index') }}" class="btn btn-secondary">← Back</a>
</div>

<div class="card" style="max-width:560px;">
<form method="POST" action="{{ url_for('add_contact') }}">

<div class="form-group">
<label for="name">Full Name *</label>
<input
type="text"
id="name"
name="name"
value="{{ form_data.get('name', '') }}"
placeholder="e.g. Alice Johnson"
required
/>
</div>

<div class="form-group">
<label for="phone">Phone Number *</label>
<input
type="tel"
id="phone"
name="phone"
value="{{ form_data.get('phone', '') }}"
placeholder="e.g. 9876543210"
required
/>
</div>

<div class="form-group">
<label for="email">Email Address *</label>
<input
type="email"
id="email"
name="email"
value="{{ form_data.get('email', '') }}"
placeholder="e.g. alice@example.com"
required
/>
</div>

<div class="form-group">
```

```
<label for="address">Address <span style="color:var(--muted); font-
weight:400;">(optional)</span></label>
<input
type="text"
id="address"
name="address"
value="{{ form_data.get('address', '') }}"
placeholder="e.g. 12 Elm Street, Delhi"
/>
</div>

<div class="form-actions">
<button type="submit" class="btn btn-primary">🖫 Save Contact</button>
<a href="{{ url_for('index') }}" class="btn btn-secondary">Cancel</a>
</div>

</form>
</div>
{% endblock %}
```

➤**App.Py:**

```
from flask import Flask, render_template, request, redirect, url_for, flash

app = Flask(__name__)
app.secret_key = "contact_mgmt_secret_key"

# — In-memory contact store ————————————————————————
# Each contact is a dict with keys: id, name, phone, email, address
contacts = [
{"id": 1, "name": "Vansh Sharma",  "phone": "9910500570", "email":
"Vansh@example.com",  "address": "12 Elm Street, Delhi"},
{"id": 2, "name": "Bob Sharma",     "phone": "9123456789", "email":
"bob@example.com",    "address": "45 Oak Avenue, Mumbai"},
{"id": 3, "name": "Carol Mehta",    "phone": "9012345678", "email":
"carol@example.com",  "address": "78 Pine Road, Bangalore"},
]
next_id = 4   # Simple auto-increment counter for new contacts
```

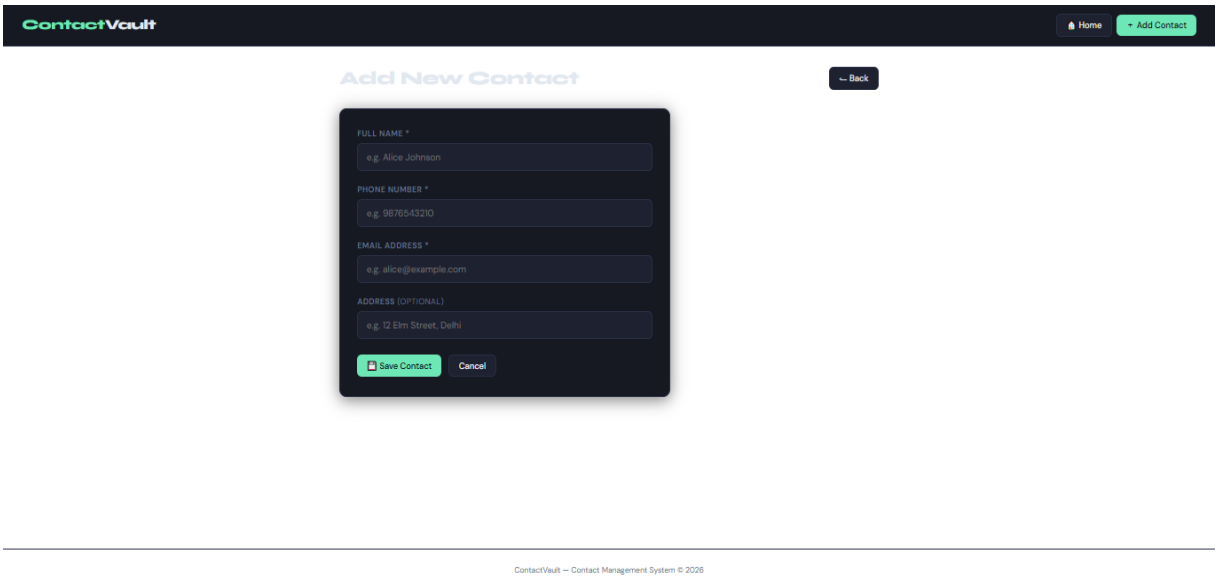
```
# — Helper ————————————————————————————————————————
def find_contact(contact_id):
"""Return the contact dict matching contact_id, or None."""
return next((c for c in contacts if c["id"] == contact_id), None)


# — Routes ————————————————————————————————————————

@app.route("/")
def index():
"""Home: list all contacts, with optional search."""
query = request.args.get("q", "").strip().lower()
if query:
results = [
c for c in contacts
if query in c["name"].lower() or query in c["phone"]
]
else:
results = contacts
return render_template("index.html", contacts=results, query=query)


@app.route("/add", methods=["GET", "POST"])
def add_contact():
"""Create a new contact."""
global next_id
if request.method == "POST":
name    = request.form.get("name", "").strip()
phone   = request.form.get("phone", "").strip()
email   = request.form.get("email", "").strip()
address = request.form.get("address", "").strip()

# — Validation ————————————————————————————————
errors = []
if not name:
errors.append("Name is required.")
if not phone:
errors.append("Phone number is required.")
elif not phone.isdigit() or len(phone) < 7:
errors.append("Phone must be numeric and at least 7 digits.")
```

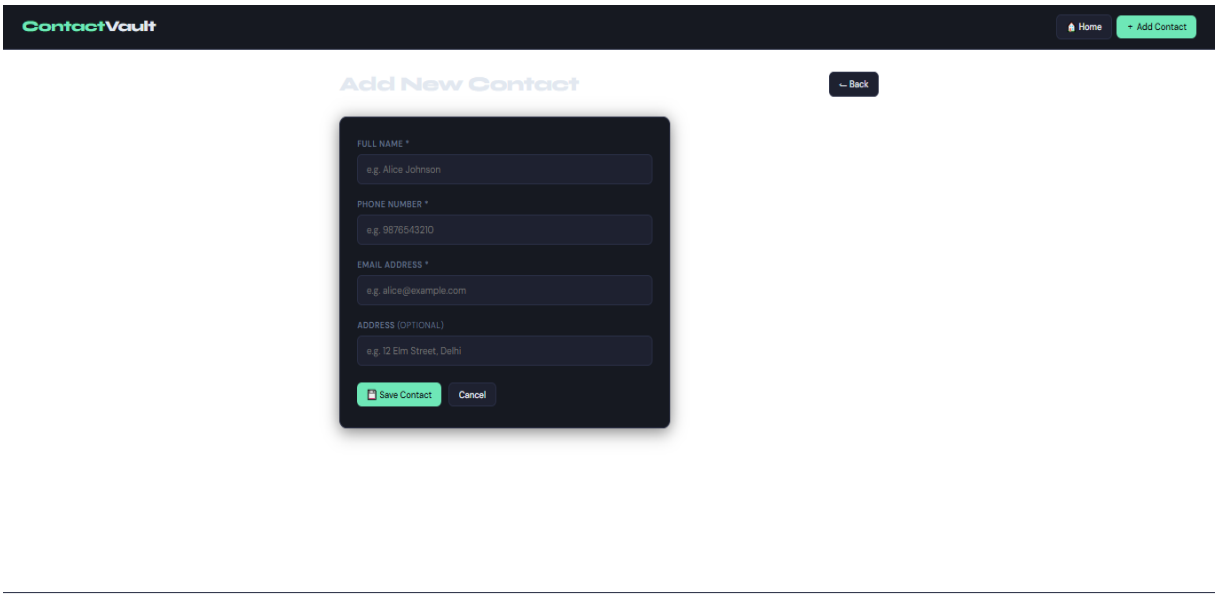
```
if not email or "@" not in email:
errors.append("A valid email address is required.")

if errors:
for err in errors:
flash(err, "error")
return render_template("add_contact.html",
form_data=request.form)

contacts.append({
"id":      next_id,
"name":    name,
"phone":   phone,
"email":   email,
"address": address,
})
next_id += 1
flash(f"Contact '{name}' added successfully!", "success")
return redirect(url_for("index"))

return render_template("add_contact.html", form_data={})


@app.route("/edit/<int:contact_id>", methods=["GET", "POST"])
def edit_contact(contact_id):
"""Update an existing contact."""
contact = find_contact(contact_id)
if not contact:
flash("Contact not found.", "error")
return redirect(url_for("index"))

if request.method == "POST":
name    = request.form.get("name", "").strip()
phone   = request.form.get("phone", "").strip()
email   = request.form.get("email", "").strip()
address = request.form.get("address", "").strip()

errors = []
if not name:
errors.append("Name is required.")
```

```
if not phone:
errors.append("Phone number is required.")
elif not phone.isdigit() or len(phone) < 7:
errors.append("Phone must be numeric and at least 7 digits.")
if not email or "@" not in email:
errors.append("A valid email address is required.")

if errors:
for err in errors:
flash(err, "error")
return render_template("edit_contact.html",
contact=contact,
form_data=request.form)

# Apply updates in-place
contact["name"]    = name
contact["phone"]   = phone
contact["email"]   = email
contact["address"] = address
flash(f"Contact '{name}' updated successfully!", "success")
return redirect(url_for("index"))

return render_template("edit_contact.html",
contact=contact,
form_data=contact)


@app.route("/delete/<int:contact_id>", methods=["POST"])
def delete_contact(contact_id):
"""Remove a contact from the list."""
global contacts
contact = find_contact(contact_id)
if contact:
contacts = [c for c in contacts if c["id"] != contact_id]
flash(f"Contact '{contact['name']}' deleted.", "success")
else:
flash("Contact not found.", "error")
return redirect(url_for("index"))
```

```
# — Entry Point ————————————————————————————————————————
if __name__ == "__main__":
app.run(debug=True)
```

❖ OUTPUT :


ContactVault
Home
+ Add Contact
Add New Contact
← Back
FULL NAME *
e.g. Alice Johnson
PHONE NUMBER *
e.g. 9876543210
EMAIL ADDRESS *
e.g. alice@example.com
ADDRESS (OPTIONAL)
e.g. 12 Elm Street, Delhi
Save Contact
Cancel
ContactVault — Contact Management System © 2026


ContactVault
Home
+ Add Contact
Add New Contact
← Back
FULL NAME *
e.g. Alice Johnson
PHONE NUMBER *
e.g. 9876543210
EMAIL ADDRESS *
e.g. alice@example.com
ADDRESS (OPTIONAL)
e.g. 12 Elm Street, Delhi
Save Contact
Cancel
ContactVault — Contact Management System © 2026